HPC PR

PR - 1

High-Performance Computing HPC A1 Practical Viva Questions (Based on Your Code)

Tuza code:

- Parallel BFS
- Parallel DFS
- OpenMP
- Graph traversal
  वर based aahe.

External almost हे questions विचारू शकतो 🔥

---

1. What is the aim of this program?

Answer:
हा program graph वर Parallel BFS आणि Parallel DFS implement करतो using OpenMP.

---

2. What is Graph?

Answer:
Nodes आणि edges चा collection म्हणजे graph.

---

3. What is BFS?

Answer:
BFS म्हणजे Breadth First Search.
हे graph ला level by level traverse करतं.

---

4. What data structure used in BFS?

Answer:
Queue वापरतात कारण BFS FIFO follow करतो.

---

5. What is DFS?

Answer:
DFS म्हणजे Depth First Search.
हे एका branch मध्ये deep जातं मग backtracking करतं.

---

6. What data structure used in DFS?

Answer:
Stack किंवा recursion वापरतात.

या code मध्ये recursion वापरलं आहे.

---

7. Difference between BFS and DFS?

BFS	DFS
Level wise traversal	Deep traversal
Queue use करतो	Stack/Recursion
FIFO	LIFO

---

8. Why OpenMP used?

Answer:
Parallel execution साठी OpenMP वापरलं आहे ज्यामुळे execution faster होतं.

---

9. What is #pragma omp parallel for?

Answer:
Loop iterations multiple threads मध्ये divide करणारा OpenMP directive.

---

10. Why #pragma omp critical used?

Answer:
Shared resources safely access करण्यासाठी critical section वापरतात.

Race condition avoid करण्यासाठी.

---

11. What is Race Condition?

Answer:
Multiple threads same data एकाच वेळी modify करतात आणि wrong result येतो.

---

12. Why visited array used?

Answer:
Same node multiple वेळा visit होऊ नये म्हणून visited array वापरतात.

---

13. Why queue used in BFS?

Answer:
BFS level-order traversal करतो त्यामुळे FIFO required असतं.

---

14. Why recursion used in DFS?

Answer:
DFS naturally recursive आहे कारण एका branch मध्ये continuously जातो.

---

15. What is Parallel BFS?

Answer:
BFS traversal parallel threads वापरून execute करणं म्हणजे Parallel BFS.

---

16. What is Parallel DFS?

Answer:
DFS traversal multiple threads वापरून execute करणं म्हणजे Parallel DFS.

---

17. What are advantages of parallel traversal?

Answer:

- Faster execution
- Better CPU utilization
- Large graphs efficiently process होतात

---

18. What problems occur in parallel graph traversal?

Answer:

- Race conditions
- Synchronization overhead
- Load balancing issues

---

19. What is Synchronization?

Answer:
Threads proper coordination मध्ये execute होतील याची खात्री करणं.

---

20. Why critical section important in this code?

Answer:
Queue आणि visited array shared आहेत त्यामुळे data corruption avoid करण्यासाठी critical section important आहे.

---

21. What is adjacency list?

Answer:
Graph store करण्याची technique जिथे प्रत्येक node चे neighbors list मध्ये store होतात.

---

22. Why vector<vector> used?

Answer:
Adjacency list implement करण्यासाठी.

---

23. Time Complexity of BFS?

Answer:
O(V+E)

V = Vertices
E = Edges

---

24. Time Complexity of DFS?

Answer:
O(V+E)

---

25. What is Shared Memory Programming?

Answer:
Multiple threads same memory access करतात त्याला shared memory programming म्हणतात.

OpenMP shared memory model वापरतो.

---

26. Why OpenMP suitable here?

Answer:
कारण हा graph traversal program shared memory system वर run होतो.

---

27. What is Thread?

Answer:
Thread म्हणजे lightweight execution unit.

---

28. What happens if critical section remove केला?

Answer:
Race condition येऊ शकतो आणि wrong output मिळू शकतो.

---

29. Can BFS and DFS outputs differ?

Answer:
हो 😄
Traversal order वेगळा असल्यामुळे outputs वेगळे येऊ शकतात.

---

30. Why parallel programming important?

Answer:
Large computations fast perform करण्यासाठी parallel programming important आहे 💧

Ho bhawa 💯 💧
Mi je questions dile na त्यात almost sagla code explanation cover झालाय.

Especially external जर:

- "हा function explain करा"
- "ही line काय करते?"
- "critical ka वापरल?"
- "parallel kasa झालं?"
- "queue ka घेतला?"
- "visited array purpose?"
- "OpenMP directive explain करा"

असं विचारलं तरी तू easily answer देऊ शकतोस 😄

Pan ata mi तुला exact function-wise quick explanation देतो. He khup useful पडेल viva मध्ये 🔥

Function-wise Explanation

1. Graph(int v)

```
Graph(int v)
{
    vertices = v;
    adj.resize(v);
}
```

काय करतो?

- Constructor आहे
- Graph चे total vertices set करतो
- Adjacency list resize करतो

Viva Answer:

"हा constructor graph initialize करतो आणि adjacency list तयार करतो."

---

2. addEdge(int u, int v)

```
adj[u].push_back(v);
adj[v].push_back(u);
```

काय करतो?

Two nodes मध्ये edge add करतो.

Important:

हा undirected graph आहे कारण दोन्ही side edge add केलंय.

Viva Answer:

"हा function graph मध्ये undirected edge add करतो."

---

3. parallelBFS(int start)

काय करतो?

Starting node पासून Parallel BFS traversal करतो.

Important lines:

Queue declaration
```
queue<int> q;
```

👉 BFS साठी queue वापरला.

---

visited array
```
vector<bool> visited(vertices, false);
```

👉 Same node repeated visit होऊ नये म्हणून.

OpenMP critical
```
#pragma omp critical
```

👉 Shared queue safely access करण्यासाठी.

OpenMP parallel for
```
#pragma omp parallel for
```

👉 Neighbor nodes parallel process करण्यासाठी.

---

4. parallelDFSUtil()

काय करतो?

Recursive Parallel DFS traversal करतो.

Important:

Recursive DFS
```
parallelDFSUtil(neighbor, visited);
```

👉 DFS recursion वापरतो.

Why critical used?

👉 Multiple threads same visited array access करतात त्यामुळे synchronization लागतो.

---

5. parallelDFS()

काय करतो?

DFS traversal start करतो.

Viva Answer:

“हा function DFS initialize करून recursive utility function call करतो.”

Main Function

Graph creation

```
Graph graph(6);
```

👉 6 vertices चा graph तयार केला.

Edge addition

```
graph.addEdge(0,1);
```

👉 Nodes connect केले.

Traversal calls

```
graph.parallelBFS(startNode);
graph.parallelDFS(startNode);
```

👉 BFS आणि DFS execute केलं.

🚨 **MOST IMPORTANT Trick Questions**

Why OpenMP used?

👉 Parallel execution faster करण्यासाठी.

Why critical section needed?

👉 Race condition avoid करण्यासाठी.

What if visited array नाही वापरला?

👉 Infinite traversal होऊ शकतो 😱

Why recursion in DFS?

👉 DFS naturally recursive आहे.

Why queue in BFS?

👉 BFS FIFO traversal follow करतो.

Ata honestly सांगतो 😄
हे समजलं na तर external काहीही विचारला तरी तू manage करू शकतोस 🔥

PR – 2

High-Performance Computing HPC A2 Practical Viva Questions (Based on Your Code)

हा practical:

- Sequential Bubble Sort
- Parallel Bubble Sort
- Sequential Merge Sort
- Parallel Merge Sort
- OpenMP

वर based आहे.

External almost हे questions विचारू शकतो 💧

BASIC QUESTIONS

1. What is the aim of this program?

Answer:
Sequential आणि Parallel Bubble Sort तसेच Merge Sort compare करणं आणि execution time analyze करणं.

---

2. What is Sorting?

Answer:
Data ascending किंवा descending order मध्ये arrange करणं म्हणजे sorting.

---

3. What is Bubble Sort?

Answer:
Adjacent elements compare करून swapping करणारा sorting algorithm.

---

4. Time Complexity of Bubble Sort?

Answer:
O(n^2)

---

5. Why Bubble Sort slow for large data?

Answer:
Repeated comparisons आणि swaps मुळे execution time जास्त लागतो.

---

6. What is Merge Sort?

Answer:
Divide-and-conquer based sorting algorithm.

---

7. Time Complexity of Merge Sort?

Answer:
O(n\log n)

---

8. Why Merge Sort faster than Bubble Sort?

Answer:
कारण Merge Sort divide-and-conquer approach वापरतो त्यामुळे comparisons कमी होतात.

---

9. What is Divide and Conquer?

Answer:
मोठा problem छोटे parts मध्ये divide करून independently solve करणं.

---

10. What is Parallel Bubble Sort?

Answer:
Bubble Sort multiple threads वापरून parallel execute करणं.

---

11. What is Parallel Merge Sort?

Answer:
Merge Sort चे subarrays multiple threads वापरून simultaneously process करणं.

OPENMP QUESTIONS

12. Why OpenMP used?

Answer:
Parallel execution faster करण्यासाठी OpenMP वापरलं आहे.

---

13. What is #pragma omp parallel for?

Answer:
Loop iterations multiple threads मध्ये divide करणारा OpenMP directive.

---

14. What is #pragma omp parallel sections?

Answer:
Different code sections simultaneously execute करण्यासाठी वापरतात.

---

15. Why parallel sections used in Merge Sort?

Answer:
Left आणि right subarrays parallel process करण्यासाठी.

---

16. What is Thread?

Answer:
Thread म्हणजे lightweight execution unit.

---

17. What is Parallel Computing?

Answer:
Multiple computations simultaneously execute करणं.

---

18. What is Speedup?

Answer:
Parallel execution मुळे program किती faster झाला हे speedup दाखवतं.

Speedup = \frac{Sequential\ Time}{Parallel\ Time}

---

19. Why execution time measured?

Answer:
Sequential आणि parallel performance compare करण्यासाठी.

---

20. Why clock() used?

Answer:
Execution time calculate करण्यासाठी.

FUNCTION-BASED QUESTIONS

21. Explain bubbleSort() function.

Answer:
हा sequential Bubble Sort implement करतो using nested loops.

---

22. Explain parallelBubbleSort() function.

Answer:
हा OpenMP वापरून Bubble Sort parallel execute करतो.

---

23. Why int first = i % 2 used?

Answer:
Odd-even sorting pattern maintain करण्यासाठी.

---

24. Explain merge() function.

Answer:
Two sorted subarrays combine करून single sorted array बनवतो.

---

25. Explain mergeSort() function.

Answer:
Recursively array divide करून merge sort perform करतो.

---

26. Explain parallelMergeSort() function.

Answer:
OpenMP parallel sections वापरून left आणि right subarrays simultaneously process करतो.

---

27. Why recursion used in Merge Sort?

Answer:
Divide-and-conquer naturally recursive approach आहे.

---

28. What is printArray() function?

Answer:
Array elements display करण्यासाठी.

---

29. Why descending order data घेतला?

Answer:
Worst-case scenario check करण्यासाठी.

---

30. Why vectors used?

Answer:
Dynamic size arrays handle करण्यासाठी.

IMPORTANT COMPARISON QUESTIONS

31. Sequential vs Parallel Execution?

Sequential	Parallel
One task at a time	Multiple tasks together
Slow	Faster

---

32. Bubble Sort vs Merge Sort?

Bubble Sort	Merge Sort
$O(n^2)$	O(n log n)
Slow	Faster
Simple	Complex

---

33. Why Merge Sort better for parallelization?

Answer:
Subarrays independently process होतात त्यामुळे parallel execution easy होतं.

---

34. Why Bubble Sort difficult to parallelize?

Answer:
Adjacent elements dependencies असल्यामुळे synchronization लागतो.

---

35. What is Scalability?

Answer:
Processors वाढवल्यावर performance improve होणं.

TRICK QUESTIONS 🚨

36. What happens if OpenMP directives remove केले?

Answer:
Program sequential execute होईल.

---

37. Which sorting algorithm better for large data?

Answer:
Merge Sort.

---

38. Which algorithm uses recursion?

Answer:
Merge Sort.

---

39. Why Merge Sort stable?

Answer:
Equal elements चा relative order maintain राहतो.

---

40. What is overhead in parallel computing?

Answer:
Thread management आणि synchronization मुळे लागणारा extra time.

---

41. Why parallel execution not always faster?

Answer:
Synchronization overhead आणि thread creation time मुळे.

---

42. What is synchronization?

Answer:
Threads proper coordination मध्ये execute होतील याची खात्री करणं.

---

43. What is load balancing?

Answer:
Tasks equally distribute करणं.

---

44. Why Merge Sort memory usage high?

Answer:
Temporary arrays तयार करावे लागतात.

---

45. What is in-place sorting?

Answer:
Extra memory कमी वापरणारा sorting algorithm.

Bubble Sort in-place आहे.

---

46. Is Merge Sort in-place?

Answer:
नाही 😄
कारण extra arrays लागतात.

---

47. Is Bubble Sort stable?

Answer:
हो 😄

---

48. Why use parallel programming?

Answer:
Execution fast करण्यासाठी आणि large computations efficiently handle करण्यासाठी.

---

49. Advantages of OpenMP?

Answer:

- Easy parallel programming
- Faster execution
- Shared memory support

---

50. Applications of Parallel Sorting?

Answer:

- Big data processing
- Scientific computing
- AI/ML
- Database systems 💧

Ho bhawa 😄💧
External khup vela direct code वरून विचारतो:

- “हा function explain करा”
- “ही line काय करते?”
- “parallel kasa झालं?”
- “odd-even logic काय आहे?”
- “merge function work kasa karto?”

Mhanun ata mi तुला code flow simple language मध्ये देतो 😄

💧 **Full Code Flow Explanation**

Step 1 — Array Create

```
vector<int> arr;
```

👉 Dynamic array तयार केला.

Step 2 — Descending Order Data

```
for(int i = n; i >= 1; i--)
{
    arr.push_back(i);
}
```

👉 100 ते 1 elements store केले.

Why?

Worst-case sorting test करण्यासाठी 😄

🔥 Bubble Sort Function

bubbleSort()

```
void bubbleSort(vector<int>& arr)
```

👉 Sequential Bubble Sort function.

Nested loops

```
for(int i = 0; i < n - 1; i++)
```

👉 Multiple passes साठी.

```
for(int j = 0; j < n - i - 1; j++)
```

👉 Adjacent elements compare करतो.

Swapping

```
if(arr[j] > arr[j + 1])
{
    swap(arr[j], arr[j + 1]);
}
```

👉 Wrong order असेल तर swap करतो.

🔥 Parallel Bubble Sort

parallelBubbleSort()

```
#pragma omp parallel for
```

👉 Loop multiple threads मध्ये divide करतो.

Important line

```
int first = i % 2;
```

👉 Odd-even sorting logic साठी.

Explain:

- Even phase → even indices compare
- Odd phase → odd indices compare

🔥 Merge Function

merge()

👉 Two sorted arrays combine करतो.

Temporary arrays

```
vector<int> L(n1), R(n2);
```

👉 Left आणि right subarrays store करतो.

Comparison

```
if(L[i] <= R[j])
```

👉 Smaller element main array मध्ये टाकतो.

🔥 Merge Sort

mergeSort()

```
int mid = (left + right) / 2;
```

👉 Array दोन parts मध्ये divide करतो.

Recursive calls

```
mergeSort(arr, left, mid);
mergeSort(arr, mid + 1, right);
```

👉 Left आणि right halves recursively sort करतो.

🔥 Parallel Merge Sort

parallel sections

```
#pragma omp parallel sections
```

👉 Multiple sections parallel execute करतो.

Section 1

```
parallelMergeSort(arr, left, mid);
```

👉 Left half parallel process.

Section 2

```
parallelMergeSort(arr, mid + 1, right);
```

👉 Right half parallel process.

🔥 Execution Time

clock()

```
clock_t start = clock();
```

👉 Start time घेतो.

```
clock_t end = clock();
```

👉 End time घेतो.

Time calculation

```
(end - start) / CLOCKS_PER_SEC
```

👉 Total execution time calculate करतो.

🚨 **MOST IMPORTANT Questions**

Why Merge Sort faster?

👉 Divide-and-conquer वापरतो.

Why parallel Merge Sort effective?

👉 Left-right subarrays independently process होतात.

Why Bubble Sort slow?

👉 Too many comparisons/swaps.

Why OpenMP used?

👉 Parallel execution साठी.

Why vectors used?

👉 Dynamic memory allocation.

Why recursive calls used?

👉 Merge Sort recursive algorithm आहे.

Why parallel Bubble Sort difficult?

👉 Adjacent element dependency असते.

Bhawa honestly 😄
He samjla na तर external code वर काहीही विचारला तरी manage होईल 🔥

PR – 3

High-Performance Computing HPC A3 Practical Viva Questions (Based on Your Code)

हा practical:

- Parallel Reduction
- Min/Max/Sum/Average
- OpenMP Reduction Clause

वर based आहे.

External almost हे questions विचारू शकतो 🔥

BASIC QUESTIONS

1. What is the aim of this program?

Answer:
Parallel Reduction वापरून minimum, maximum, sum आणि average calculate करणं.

---

2. What is Parallel Reduction?

Answer:
Multiple values combine करून single result तयार करणं म्हणजे reduction.

Example:
sum, min, max.

---

3. Why OpenMP used?

Answer:
Parallel execution faster करण्यासाठी OpenMP वापरलं आहे.

---

4. What is Reduction Clause?

Answer:
Reduction clause प्रत्येक thread साठी private copy तयार करतो आणि शेवटी results combine करतो.

---

5. What is #pragma omp parallel for?

Answer:
Loop iterations multiple threads मध्ये divide करणारा OpenMP directive.

---

6. What is reduction(min:min_val)?

Answer:
Parallel threads मधून minimum value safely calculate करण्यासाठी.

---

7. What is reduction(max:max_val)?

Answer:
Maximum value parallel way ने calculate करण्यासाठी.

---

8. What is reduction(+:sum)?

Answer:
Parallel sum calculation साठी.

---

9. Why reduction clause important?

Answer:
Race condition avoid करण्यासाठी.

---

10. What is Race Condition?

Answer:
Multiple threads same variable modify करतात आणि wrong result येतो.

CODE EXPLANATION QUESTIONS

11. Why vector used?

Answer:
Dynamic size array handle करण्यासाठी.

---

12. Why srand(time(0)) used?

Answer:
Random numbers generate करण्यासाठी seed initialize करतो.

---

13. What does rand() % 100 + 1 do?

Answer:
1 ते 100 random numbers generate करतो.

---

14. Why min_val and max_val initialized with arr[0]?

Answer:
Comparison start करण्यासाठी initial value लागते.

---

15. Explain Min operation loop.

Answer:
प्रत्येक thread elements compare करून minimum value find करतो.

---

16. Explain Max operation loop.

Answer:
Maximum element parallel way ने find करतो.

---

17. Explain Sum operation.

Answer:
सगळ्या elements ची addition parallel threads वापरून करतो.

---

18. Explain Average operation.

Answer:
Average = Sum / Number of elements

Average = \frac{Sum}{n}

---

19. Why clock() used?

Answer:
Execution time measure करण्यासाठी.

---

20. What is Execution Time?

Answer:
Program execute होण्यासाठी लागलेला time.

IMPORTANT CONCEPT QUESTIONS

21. What is Parallel Computing?

Answer:
Multiple computations simultaneously execute करणं.

---

22. What is Thread?

Answer:
Lightweight execution unit.

---

23. What is Synchronization?

Answer:
Threads proper coordination मध्ये execute होतील याची खात्री करणं.

---

24. Why reduction better than critical section?

Answer:
Reduction optimized आणि faster असतो.

Critical section slower असतो कारण एकावेळी एक thread execute होतो.

---

25. Reduction vs Critical Section?

Reduction	Critical Section
Faster	Slower
Optimized	One thread at a time

---

26. What is Shared Memory?

Answer:
Multiple threads same memory access करतात.

---

27. What is Scalability?

Answer:
Processors वाढवल्यावर performance improve होणं.

---

28. Why parallel execution faster?

Answer:
Multiple threads simultaneously work करतात त्यामुळे execution time कमी होतो.

---

29. What is Overhead?

Answer:
Thread management आणि synchronization मुळे लागणारा extra time.

---

30. Applications of Parallel Reduction?

Answer:

- Big data processing
- Scientific computing

- AI/ML
- Statistical analysis

TRICK QUESTIONS 🚨

31. What happens if reduction clause remove केला?

Answer:
Race condition येईल आणि wrong output मिळू शकतो.

---

32. Why average operation parallel नाही?

Answer:
Average calculation simple division आहे त्यामुळे separate parallel loop नाही.

---

33. Why random numbers used?

Answer:
Different test cases generate करण्यासाठी.

---

34. Why OpenMP suitable here?

Answer:
Shared memory parallel programming साठी.

---

35. Can reduction perform other operations?

Answer:
हो 😄
sum, multiplication, min, max etc.

---

36. What is Load Balancing?

Answer:
Tasks threads मध्ये equally distribute करणं.

---

37. What is Thread Safety?

Answer:
Multiple threads safely shared data access करतील याची खात्री करणं.

---

38. Why reduction clause thread-safe आहे?

Answer:
प्रत्येक thread ला private variable मिळतो.

---

39. What is the advantage of OpenMP?

Answer:
Easy parallel programming आणि faster execution.

---

40. Why parallel programming important?

Answer:
Large computations fast execute करण्यासाठी 💧

Ho bhawa 😄💧
Ya practical madhye external almost code explanation pan vicharel:

- "Reduction clause explain करा"
- "ही line काय करते?"
- "random numbers ka generate kele?"
- "parallel kasa झालं?"
- "sum safely kasa calculate झाला?"

Mhanun ata mi तुला full code flow simple language मध्ये देतो 😄

💧 **Full Code Explanation**

1. Input Size

```
cin >> n;
```

👉 User कडून total elements घेतो.

---

2. Vector Creation

```
vector<int> arr(n);
```

👉 Dynamic array तयार केला.

---

3. Random Seed

```
srand(time(0));
```

👉 Random number generator initialize करतो.

---

4. Random Numbers Generate

```
arr[i] = rand() % 100 + 1;
```

👉 1 ते 100 random numbers generate करतो.

---

5. Array Print

```
cout << arr[i] << " ";
```

👉 Generated list display करतो.

---

6. Initialize Variables

```
int min_val = arr[0];
int max_val = arr[0];
int sum = 0;
```

👉 Initial values set केल्या.

Why arr[0]?

Comparison start करण्यासाठी.

🔥 MIN Operation

OpenMP Reduction

```
#pragma omp parallel for reduction(min:min_val)
```

काय करतो?

- Loop parallel execute करतो
- प्रत्येक thread स्वतःचा minimum शोधतो
- शेवटी combine होतो

Comparison

```
if(arr[i] < min_val)
```

👉 Smaller value मिळाली तर update करतो.

🔥 MAX Operation

```
#pragma omp parallel for reduction(max:max_val)
```

👉 Parallel maximum calculation.

Comparison

```
if(arr[i] > max_val)
```

👉 Maximum value update करतो.

🔥 SUM Operation

```
#pragma omp parallel for reduction(+:sum)
```

👉 Parallel addition.

Addition

```
sum += arr[i];
```

👉 Elements add करतो.

🔥 **AVERAGE Operation**

```
avg = (double)sum / n;
```

👉 Average calculate करतो.

Formula:
Average = \frac{Sum}{n}

🔥 **Execution Time**

Start Time

```
clock_t start = clock();
```

👉 Execution start time घेतो.

End Time

```
clock_t end = clock();
```

👉 End time घेतो.

Time Calculation

```
(end - start) / CLOCKS_PER_SEC
```

👉 Total execution time calculate करतो.

🚨 **MOST IMPORTANT Viva Questions**

Why reduction clause used?

👉 Race condition avoid करण्यासाठी.

Why OpenMP used?

👉 Parallel execution fast करण्यासाठी.

What happens without reduction?

👉 Wrong results येऊ शकतात.

Why random numbers used?

👉 Different inputs testing साठी.

Why parallel execution faster?

👉 Multiple threads simultaneously work करतात.

Why average directly calculate केला?

👉 कारण sum already मिळालेला आहे 😄

Difference between min/max/sum reduction?

Reduction	Purpose
reduction(min:var)	Minimum find
reduction(max:var)	Maximum find
reduction(+:var)	Sum calculate

Bhawa honestly 😄
He samjla na तर external code वर काहीही विचारला तरी manage होईल 💧

PR – 4

High-Performance Computing HPC A4 Practical Viva Questions (Based on Your Code)

हा practical:

- Parallel Vector Addition
- Parallel Matrix Multiplication
- OpenMP Parallelization

वर based आहे.

External almost हे questions विचारू शकतो 💧

BASIC QUESTIONS

1. What is the aim of this program?

Answer:
Parallel Vector Addition आणि Matrix Multiplication using OpenMP implement करणं.

---

2. What is Vector Addition?

Answer:
Two vectors चे corresponding elements add करणं.

Example:
A[i] + B[i]

---

3. What is Matrix Multiplication?

Answer:
Rows आणि columns multiply करून resultant matrix तयार करणं.

---

4. Why OpenMP used?

Answer:
Parallel execution fast करण्यासाठी.

---

5. What is #pragma omp parallel for?

Answer:
Loop iterations multiple threads मध्ये divide करणारा OpenMP directive.

---

6. Why vector addition suitable for parallelization?

Answer:
प्रत्येक element independently compute होतो त्यामुळे parallel execution easy आहे.

---

7. Why matrix multiplication suitable for parallelization?

Answer:
प्रत्येक matrix element independently calculate होतो.

---

8. What is Parallel Computing?

Answer:
Multiple computations simultaneously execute करणं.

---

9. What is Thread?

Answer:
Lightweight execution unit.

---

10. What is Speedup?

Answer:
Parallel execution मुळे computation किती fast झाला हे speedup दाखवतं.

Speedup = \frac{Sequential\ Time}{Parallel\ Time}

VECTOR ADDITION QUESTIONS

11. Explain vectorAddition() function.

Answer:
हा function दोन vectors create करून parallel way ने addition करतो.

---

12. Why vectors A, B, C used?

Answer:
A आणि B input vectors आहेत.
C resultant vector आहे.

---

13. Why random numbers used?

Answer:
Different input values generate करण्यासाठी.

---

14. Explain this line:

```
C[i] = A[i] + B[i];
```

Answer:
Corresponding vector elements add करून result C मध्ये store करतो.

---

15. Why parallel for used in vector addition?

Answer:
प्रत्येक iteration independent आहे त्यामुळे multiple threads simultaneously execute करू शकतात.

MATRIX MULTIPLICATION QUESTIONS

16. Explain matrixMultiplication() function.

Answer:
हा function two matrices multiply करून resultant matrix तयार करतो.

---

17. Formula for matrix multiplication?

C[i][j] = \sum A[i][k] \times B[k][j]

---

18. Why three loops used in matrix multiplication?

Answer:

- i → rows
- j → columns
- k → multiplication and summation

---

19. Explain this line:

```
C[i][j] += A[i][k] * B[k][j];
```

Answer:
Corresponding row-column elements multiply करून sum करतो.

---

20. What is collapse(2)?

```
#pragma omp parallel for collapse(2)
```

Answer:
Nested loops combine करून parallel execution improve करतो.

---

21. Why collapse(2) used?

Answer:
Better workload distribution आणि improved parallel performance साठी.

---

22. What is resultant matrix?

Answer:
Matrix multiplication नंतर मिळालेला final matrix.

CODE EXPLANATION QUESTIONS

23. Why clock() used?

Answer:
Execution time measure करण्यासाठी.

---

24. What is Execution Time?

Answer:
Program execute होण्यासाठी लागलेला time.

---

25. Why srand(time(0)) used?

Answer:
Random seed initialize करण्यासाठी.

---

26. Why vectors dynamic आहेत?

Answer:
Dynamic memory allocation साठी.

---

27. What is shared memory programming?

Answer:
Multiple threads same memory access करतात.

---

28. Why OpenMP suitable here?

Answer:
Shared memory parallel programming easy होतं.

---

29. What is scalability?

Answer:
Processors वाढवल्यावर performance improve होणं.

---

30. What is load balancing?

Answer:
Tasks threads मध्ये equally distribute करणं.

IMPORTANT COMPARISON QUESTIONS

31. Sequential vs Parallel Vector Addition?

Sequential	Parallel
One element at a time	Multiple elements simultaneously

---

32. Sequential vs Parallel Matrix Multiplication?

Sequential	Parallel
Slow	Faster
Single thread	Multiple threads

---

33. Why matrix multiplication computationally expensive?

Answer:
Large number of multiplications आणि additions लागतात.

---

34. Why HPC useful for matrix operations?

Answer:
Large computations faster perform करता येतात.

TRICK QUESTIONS 🚨

35. What happens if OpenMP directive remove केला?

Answer:
Program sequential execute होईल.

---

36. Why vector addition easy to parallelize?

Answer:
Iterations independent आहेत.

---

37. What happens if dimensions mismatch झाले?

Answer:
Matrix multiplication possible होणार नाही.

---

38. Why matrix multiplication important?

Answer:
AI/ML, graphics, scientific computing मध्ये वापरतात.

---

39. Why execution time compare करतो?

Answer:
Parallel performance analyze करण्यासाठी.

---

40. Applications of vector and matrix operations?

Answer:

- AI/ML

- Graphics
- Scientific simulations
- Big data processing

🔥 CODE FLOW EXPLANATION

vectorAddition()

Step 1:
```
vector<int> A(n), B(n), C(n);
```

👉 Three vectors तयार केले.

Step 2:
```
A[i] = rand() % 100;
```

👉 Random values initialize केल्या.

Step 3:
```
#pragma omp parallel for
```

👉 Loop multiple threads मध्ये divide केला.

Step 4:
```
C[i] = A[i] + B[i];
```

👉 Vector addition perform केली.

---

matrixMultiplication()

Step 1:

Matrices initialize केल्या.

Step 2:

```
#pragma omp parallel for collapse(2)
```

👉 Nested loops parallel execute केले.

Step 3:

```
C[i][j] += A[i][k] * B[k][j];
```

👉 Matrix multiplication formula apply केली.

Step 4:

Resultant matrix print केला.

🚨 **MOST IMPORTANT Viva Questions**

Why collapse(2) used?

👉 Nested loops efficiently parallelize करण्यासाठी.

Why OpenMP used?

👉 Faster parallel execution साठी.

Why matrix multiplication expensive?

👉 Multiple nested loops असल्यामुळे.

Why vector addition faster?

👉 Independent operations असल्यामुळे.

Why parallel programming important?

👉 Large computations fast solve करण्यासाठी 💧